



TAYLOR'S UWE DUAL AWARDS PROGRAMMES
SEPTEMBER 2025 SEMESTER
BIG DATA TECHNOLOGIES
(ITS66904)

Assignment 1-Individual (30%)

DUE DATE: 28th November 2025 via MyTIMeS (11:59 pm)



STUDENT DECLARATION

- 1. I confirm that I am aware of the University's Regulation Governing Cheating in a University Test and Assignment and of the guidance issued by the School of Computing and IT concerning plagiarism and proper academic practice, and that the assessed work now submitted is in accordance with this regulation and guidance.*
- 2. I understand that, unless already agreed with the School of Computing and IT, assessed work may not be submitted that has previously been submitted, either in whole or in part, at this or any other institution.*
- 3. I recognise that should evidence emerge that my work fails to comply with either of the above declarations, then I may be liable to proceedings under Regulation.*

Student Name	Student ID	Date	Signature	Score
Retaj Ahmed [REDACTED] [REDACTED]	[REDACTED]	27/11/2025	[REDACTED]	

Table of Contents

Table of Contents.....	1
1. Introduction.....	2
Why Fraud Now Demands Big Data Solution.....	3
2. Literature Review and Critical Analysis of Existing Solutions.....	4
2.1 Literature review of existing Big Data solutions.....	4
2.1.1 Apache Flink Stream Processing.....	4
2.1.2 Apache Spark Structured Streaming.....	5
2.1.3 Kafka Streams.....	6
2.2 Existing Solutions Strengths and Limitations.....	8
2.2.1 Apache Flink.....	8
2.2.2 Apache Spark.....	9
2.2.3 Kafka Streams.....	10
2.2.4 Comparative Analysis between the Three Solutions for Fraud Detection.....	11
3. Proposed Big Data Solution.....	13
3.1 Architecture Diagram.....	13
3.1.1 Architecture Diagram Breakdown.....	13
3.1.2 Technologies Breakdown.....	16
3.2 Components Interaction.....	19
3.2.1 Workflow Diagram.....	19
4. Challenges, findings and recommendations.....	21
References.....	22
Marking Rubrics.....	24

1. Introduction

Financial fraud is one of the most severe and expensive challenges facing banking and FinTech fields globally nowadays. The Association of Certified Fraud Examiners (2024) estimates that organizations lose 5% of annual revenues to fraud, equating to over USD 4.7 trillion annually and the financial industry was disproportionately impacted due to its central position in the transactional ecosystems. In reaction, banks are increasingly resorting to Big Data technologies to detect and prevent fraudulent activities in real time. For the given case study, a major international bank needs a high-velocity Big Data platform that would identify fraudulent transactions and have a strict latency limit of less than 500 milliseconds. This limitation represents the practicalities of today's payment systems, where a delay over half a second can cause an irreversible loss and poor customer experience.

This demand places a huge burden on two fundamental aspects of Big Data: velocity, speed at which data can be produced and processed, and volume, amount of transactional data which can often run into millions of events per second during peak times. Most traditional batch-based fraud detection systems, by definition, process transactions once they have occurred, whereas in real-time systems, data must be ingested, processed and acted upon as it arrives, which requires stream processing architectures designed to execute in a continuous low-latency manner (ReenBit-admin, 2025). Through the report, we will critically analyze existing scholarly literature on Big Data for real-time fraud detection, and propose a robust, scalable architecture that meets the bank's demanding performance and accuracy requirement based on the analysis and insights collected.

Why Fraud Now Demands Big Data Solution

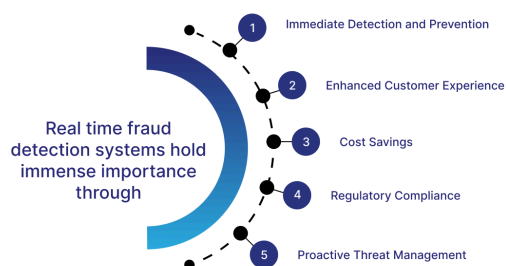


Figure 1 - Essential Capabilities of a Modern Real-Time Fraud Detection System

The growing risk of financial fraud in the digital age is a product of a combination of factors that lie in the essentials of modern banking development. The growth of internet transactions, mobile payments, and international e-commerce has resulted in a large attack surface which is continuously tested by highly developed and evolved attackers (Moon et al., 2022). Contrary to the past fraud, which mostly entailed isolated, basic frauds, modern schemes are described by their speed, scale, and adaptability. In order to carry out thousands of low-value, testing, automated bots; criminals use fraudulent identity details to open new accounts, and organize rings where frauds are distributed across various entities to escape detection by simple, rule-based systems. Such schemes produce huge amounts of data and take advantage of the time delays in the conventional systems in which a few seconds of delay is sufficient to allow a fraudulent transaction to be irreversibly executed (SIF, 2022).

Therefore, a Big Data system with high detection potentially cannot be restricted to traditional batch processing. It demands a real-time, sub-second processing ability to analyze and handle transactions in real time. Moreover, it should be scalable to absorb and process billions of events per day, intelligent and stateful, able to go beyond fixed rules to use ML models that can extract subtle, non-obvious patterns. Most importantly, the system should have the capability of maintaining and constantly updating a contextual memory of user action including spending patterns, location history, and device usage in order to immediately identify abnormal actions, not corresponding to an established norm, to identify high-level fraud which otherwise would have been legitimate within a vacuum.

2. Literature Review and Critical Analysis of Existing Solutions

2.1 Literature review of existing Big Data solutions

Through this part, each section will be highlighting an existing big data solution and referring to one main scholarly source for that technology.

2.1.1 Apache Flink Stream Processing

Referring to Sarbaree Mishra (2023) scholarly article: "Scaling Rule Based Anomaly and Fraud Detection and Business Process Monitoring Through Apache Flink", Apache Flink is a complex, resilient open-source framework particularly designed with stateful stream processing in the realm of real-time fraud detection. Its distributed architecture is designed to execute in clusters, hence it is horizontally scalable by merely adding more nodes to handle rising data loads, which is essential in monitoring high-velocity financial transactions. The most common programming languages used by developers are Java and Scala to create complex business logic using Flink `DataStream` and `DataSet` APIs. Although its core strength is programmatic processing, Flink provides a uniform query language layer through its Table API and SQL support, enabling declarative query streaming data, which streamlines the creation of specific analytical patterns. The foundational data storage and processing model is streaming model; a continuous, unlimited data stream processed on an event-by-event basis with minimal latency (sub-100 ms). This is supported by a strong state management system, where intermediate state (e.g., user session data, rolling counters) is persisted in configurable backends such as RocksDB. Flink has a wide data type support including basic data types, arrays, and composite types like POJOs and Scala case classes, enabling it to natively process structures, semi-structured, and unstructured data from various resources such as Kafka, Kinesis and log files. Its main features constitute an in-depth fraud detection tool. Stateful Processing is the most important for monitoring user behaviour over time, including maintaining running totals of transaction amounts or event sequences to detect suspicious patterns. This is supported by a strong fault tolerance providing asynchronous checkpointing which generates consistent snapshots of the application state, thus guaranteeing exactly-once processing semantics and preventing data loss in the event of node failure. To detect more complex schemes, Flink's Complex Event Processing (CEP) library offers a specialized model of recognizing multi-step patterns within the event stream, for instance, a sequence of small payments followed by a large withdrawal. Moreover, Event Time and Windowing gives the ability to analyze data within a timeframe through powerful mechanisms such as tumbling, sliding and session windows which is essential in detecting fraudulent activities that transpire within a specific period. The key advantages of this method are detection of complex, contextual fraud in real-time with

very low latency while being highly scalable processing billions of events per second. On the other hand, its limitations are operational complexity in managing clusters, complexity of designing, tuning and maintaining complex rules and models underlying a production scale detection system. This balance between ultra-low latency, rich state management, expressive pattern matching, and production-grade reliability makes Flink especially favored in high-stakes, high-throughput application areas like real-time payment screening, insurance claims verification, and cross-border anti-money laundering (AML) monitoring, where both speed and contextual fidelity are essential (Ververica, 2024).

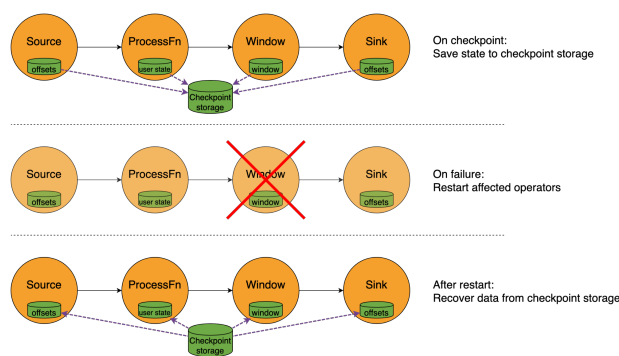


Figure 2 - Apache Flink's Fault Tolerance Mechanism via Asynchronous Checkpointing

The diagram illustrates Flink's checkpointing mechanism capturing consistent snapshots of the application states such as source offsets, user state in operators, window state, sink offsets all stored in durable storage. Fault tolerance and exactly-once processing semantics are facilitated by this process, which guarantees no data loss and state consistency even in the presence of node failures.

2.1.2 Apache Spark Structured Streaming

According to Bhargav Vadgama (2025) research paper: “Real-Time Fraud ML on Spark Structured Streaming: Micro-Batch vs. Continuous Processing”, Apache Spark Structured Streaming is the core, stateful stream processing and ML interface engine within the real-time fraud detection system. Its distributed architecture is intended for seamless integration within a modular data pipeline using Apache Kafka as a high-throughput ingestion layer and emit processed outputs to sinks such as PostgreSQL or Elasticsearch used for alerts and visualizations. The mainly used programming languages are Scala for critical high performance applications and PySpark as a robust machine learning library ecosystem, both based on high-level DataFrame and Dataset APIs. A unified query language layer is provided through Spark SQL and Table API, allowing developers to run declarative, SQL-like queries on continuous data streams. Its data storage and processing model is uniquely flexible having two execution modes: the default Micro-Batch processing model which executes data in small, fault tolerant intervals (1-2 sec), and Continuous processing model which executes data record-by-record achieving low end-to-end latency (20 ms). It is also natively compatible with various data types, including structures JSON and Avro enabling the ingestion of complex transactional metadata from various sources.

Its main features form a full toolkit for real-time analytics; it offers exact-once processing semantics evaluated through robust checkpoint and write-ahead logs, ensuring no data is lost or duplicated even in case of failures. Powerful windowing operations enabling the execution of complex time-bound aggregation (e.g., calculating transaction numbers per user in the last 10 minutes). Stateful Stream processing enables the maintenance and updates of specific user context (e.g., running transaction averages) essential to detect behavioral anomalies. Its unified engine allows the seamless integration of pre-trained ML models within the streaming pipeline, enabling real-time fraud scoring. The major benefits of this approach are its high throughput, fault tolerance and capability to employ a single programming model that supports both batch and stream processing; simplifying developments and maintenance. The limitations are characterized by the tradeoff between its two processing modes, micro-batch provides full transformational power and production grade stability, but at the expense of higher latency (400-700 ms). Despite the fast speed and lower latency of the continuous processing mode, it has limited fault tolerance, restricted to stateless operations and still considered experimental for critical mission deployments; therefore its architectural choice is a critical design decision.

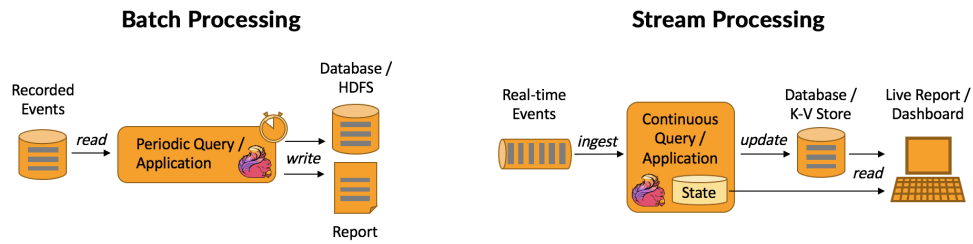


Figure 3 - Processing Models Behind Spark's Streaming Modes

The diagram clearly illustrates the difference between batch processing and stream processing in the unified engine, allowing both historical and real-time processing using the same programming model. The stateful stream processing feature, critical to maintain user context and the detection of behavioral anomalies, is directly associated with the stateful component in the stream processing flow. The two execution modes (micro-batch for periodic processing and continuous for low latency, record-by-record execution) are reflected by the Continuous Query/Application consuming real-time events. The design portrays the core flexibility and state management functionalities that enables Spark to be a suitable solution for real-time fraud detection.

2.1.3 Kafka Streams

The proposed system by Kishore (2025) in his research paper “AI-DRIVEN REAL-TIME FRAUD DETECTION USING KAFKA STREAMS IN FINTECH” highlights Apache Kafka as a lightweight, embedded stream-processing model that is uniquely adopted for scalable fraud detection systems in FinTech, with ultra-low-latency requirements. It is a cloud-native, microservices-based architecture, where services as data ingestion, stream processing, and machine learning scoring are hosted in Docker containers, coordinated by Kubernetes, and communicating asynchronously via Kafka topics to maintain loose coupling and fault tolerance. The programming language ecosystem includes Python to implement Kafka producers, Flask based RESTful scoring microservices, and the core stream processing logic is developed using Java or Scala. The framework does not use a declarative query language; meanwhile, data transformations are written in a developer-friendly, procedural way using the Kafka Streams Domain-Specific Language (DSL) and Processor API. The data storage and processing model is a unified and distributed approach; Kafka durable, partitioned and replicated commit log acts as a persistent, high throughput message bus. Kafka Stream will stream stateful, event-by-event operations maintaining intermediate results in embedded fault-tolerant state stores like RocksBD for aggregations and windowing operations. The system is intended to process structured transactional data types, mainly in JSON objects with transaction ID, user ID, amount, geolocation, and timestamps fields. Its main characteristics form a powerful, built-in toolkit of real-time analytics; it offers exact-once processing semantics guaranteed by Kafka transactional procedures and idempotent writes with no data loss or duplication even in failure events. Stateful stream processing with built-in key value stores to perform real-time feature engineering directly within the pipeline such as continuous updates of user rolling average transaction amount or counting login attempts from a specific IP address. Seamless integration with external services enables the streaming topology to synchronously query containerized ML models through REST APIs for fraud probability scoring; combining stream processing low latency with complex ML models predictive power. The key benefits of this solution are its outstanding performance reflected by consistent sub-250 ms end-to-end latency, throughput of over 500 transactions per second, its operational simplicity due to the cohesiveness of the storage and processing layers, and its modular design allowing independent scaling and model updates. Limitation focus on structured, tabular data and the absence of support for unstructured data sources, reliance on high performance yet opaque ML models such as XGBoost which requires future integration with explainability frameworks to comply with regulations and operational trust.

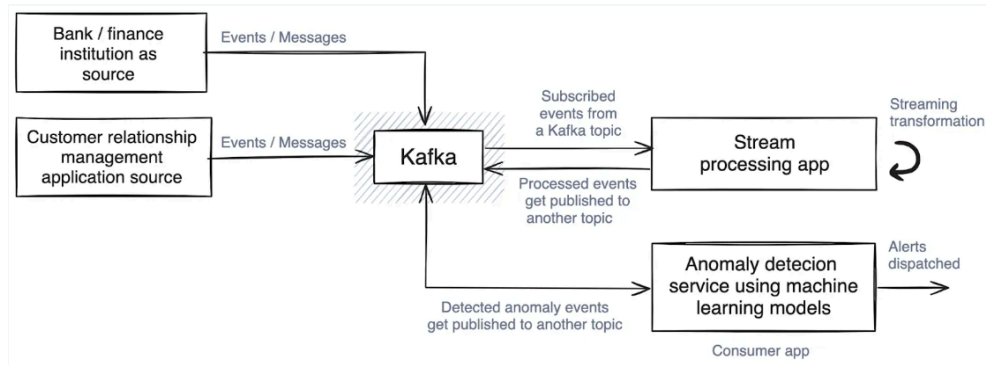


Figure 4 - End-to-End Architecture for Real-Time Fraud Detection with Kafka Streams and External ML Services

The diagram ideally represents the microservices-based architecture, demonstrating how events are ingested from sources (bank) into Kafka and processed by specialized stream processing applications. The flow of proceed events and detected anomaly events to various Kafka topics illustrates the system's modularity, loosely coupled, allowing services as ML-based anomaly detector to run independently. The diagram depicts the dispatched alerts, achieving low latency for real-time fraud detection within the Kafka-centric pipeline.

2.2 Existing Solutions Strengths and Limitations

2.2.1 Apache Flink

Strengths:

- **Better Velocity Handling:** Flink has a real stream-processing model which is its core strength. It uses one-by-one event processing and hence ultra-low latencies (sub-100 ms). This is essential in pre-authorization fraud control in which a one millisecond delay can result in a fraudulence transaction being accepted.
- **Strong State Management at Volume:** Flink killer volume feature is in its fault-tolerant, scalable state management. It can sustain complex, evolving user profiles (e.g. rolling transaction averages) within massive load without the need for external databases. Its robust checkpointing mechanism ensures fault tolerance with exactly-once preventing any data loss or duplication during failures. This enables accurate and scalable context-aware detection and it directly meets the requirement of processing large volumes of data with rich and stateful logic.
- **Complex Event Processing (CEP):** Its native CEP library can detect complex, multi-event patterns of fraud (e.g., rapid transactions across geographies) in real-time, which is required to detect coordinated attacks which occur at high velocities.

Limitations:

- **Operational Complexity:** Managing a distributed Flink cluster increases operational overhead (managing clusters, backends, performance tuning and cost) compared to more lightweight frameworks. This complexity could be a barrier and make it more expensive to maintain the high-volume pipelines.
- **Ecosystem Maturity:** As it continues to grow, Flink does not have as mature or integrated ecosystem for machine learning compared to Spark.

Paper : <https://doi.org/10.63282/3050-9416.ijaibdcms-v4i1p111>.

2.2.2 Apache Spark

Strengths:

- **Robust Analytical Processing on High-Volume Data:** Spark's greatest strength is that it is based on micro-batch architecture integrated with a rich ecosystem (Spark SQL, MLlib). This allows the usage of the same code and APIs for historical data analysis and real-time inferences, making it a simpler architecture for developers. It enables complex feature engineering (e.g. computing 30-minute transaction velocity deviations) of large amounts of data. This results in slightly better precision/recall shown in Vadgama study, since the model uses more rich contextual features.
- **Fault Tolerance for Volume-Intensive Workloads:** By using offset checkpointing to achieve exactly-once semantics in micro-batch mode; ensures both data integrity and reliable processing of billions of events.
- **Developer Familiarity:** Its batch-like programming language reduces the complexity barrier that developers have to structured, high volume data pipelines.

Limitations:

- **Inherent Latency through Micro-Batching:** The fundamental design limitation increased latency (400-700 ms); a direct trade-off of its analytical power. This delay can be intolerable to the most sensitive fraud applications, for instance, blocking a payment at the gateway.
- **Limited Real-Time Capability:** In the Continuous Processing mode, with a low latency (1ms), it is also highly restricted to stateless operations. This renders its incompatibility with the stateful aggregations and windowing that lie at the core of successful fraud detection, and renders its effectivity to high-velocity, context-sensitive analysis.

Paper: <https://utilitasmathematica.com/index.php/Index/article/view/2786>

2.2.3 Kafka Streams

Strengths:

- **Optimized for Velocity and Low-latency:** Kafka Streams being an embedded library avoids network traversals and processes alongside the data. This gives it the lowest end-to-end latency (less than 250 ms), and thus the fastest fraud screening cycles.
- **Simple Scalability for Volume:** It uses native partitioning and replication of Kafka itself for scaling and fault tolerance. When the volume of data increases, you scale the Kafka cluster and the stream processing scales with it seamlessly, providing a simple model of horizontal scaling.
- **Decoupled ML lifecycle:** Decoupling the ML model as a separate REST service enables retraining and testing the model independently of the high-velocity streaming topology.

Limitations:

- **Weaker Stateful Processing Capabilities:** Although it has state stores (e.g. RocksDB), its stateful processing is less expressive and less powerful than Flink. It is not optimized to deal with high complexity and multi-stream joins or advanced windowing that are needed to handle the sophisticated fraud patterns over large and interconnected datasets.
- **Embedded Model; Limited Scope:** It is lightweight, embedded model; not qualified as a full-fledged cluster manager. It is most appropriate as a specific edge filter in a larger ecosystem. To detect fraud, which involves deep, multi-source contextual analysis, it might be necessary to combine it with a more powerful engine such as Flink or Spark.

Paper: <https://doi.org/10.12732/ijam.v38i6s.433>.

2.2.4 Comparative Analysis between the Three Solutions for Fraud Detection

Table 1 - Existing Solutions Comparative Analysis

Feature	Apache Flink 	Apache Spark 	Kafka Stream 
Architectural Paradigm	Stream Processor (event-by-event)	- Micro-Batch Processor - Continuous Processing	Embedded Stream Processing Library
Latency	Sub - 100 ms (low)	- Micro-Batch: 400-700 ms (high) - Continuous: ~1 ms (stateless-only)	Sub-250 ms (Optimized end-to-end)
High Volume Handling	High Scalability. High throughput. Stateful workloads.	High Scalability. Optimized for large-scale data processing. Efficient resource utilization in micro-batches.	Scalable. Best for high-volume, simple logic.
High Velocity Handling	Event driven model provide immediate reaction without delays	Micro-batching introduce inherent delays. Continuous mode is fast, but functionality limited	Embedded design and lightweight API minimizes processing overhead
State Management	Powerfull and Flexible Fault tolerant. Ideal for complex user profiles.	Robust in micro-batch supporting stateful operations. Not provided in Continuous mode.	Basic Local state stores. Good for per-key aggregations. Less suited for complex multi-stream states.
Complex Analysis and Machine	Strong Native CEP for	Strongest Seamless integration	Limited Relies on external ML

Learning	pattern matching Limited ML integration	with Spark MLlib for rich, complex feature engineering.	microservices Not applicable for complex in-stream analytics.
Fault-Tolerance and Semantics	Exactly-Once. Asynchronous, incremental checkpointing with minimal performance impact.	Exactly-Once in Micro-Batch. At-Least-Once in Continuous mode.	Exactly-once. Via Kafka transactions and offset tracking.
Operational Complexity	High Requires cluster management and tuning expertise.	High Requires cluster management. Familiarity might ease operations.	Low Embedded Library, no separate clusters.
Key Limitation	Operational complexity; learning cluster management.	latency trade-off of its micro-batch architecture for stateful processing.	Not designed for complex, multi-stream joins or advanced analytics
Ideal Fraud Detection Use case	Real-time payment screening, complex multi-event pattern detection, and contexts requiring rich, stateful context with lowest latency.	Deep contextual analysis where feature richness and model accuracy are prioritized over latency	High-speed, pre-authorization filtering, transaction blocking, and edge-level risk scoring where speed and operational simplicity are critical

3. Proposed Big Data Solution

By referring to the literature review analysis in the previous section - question 1: the proposed solution is a combination of two of the existing big data solutions (Apache Flink and Kafka Streams) with Apache Kafka, aiming to enhance the overall process of fraud detection.

3.1 Architecture Diagram

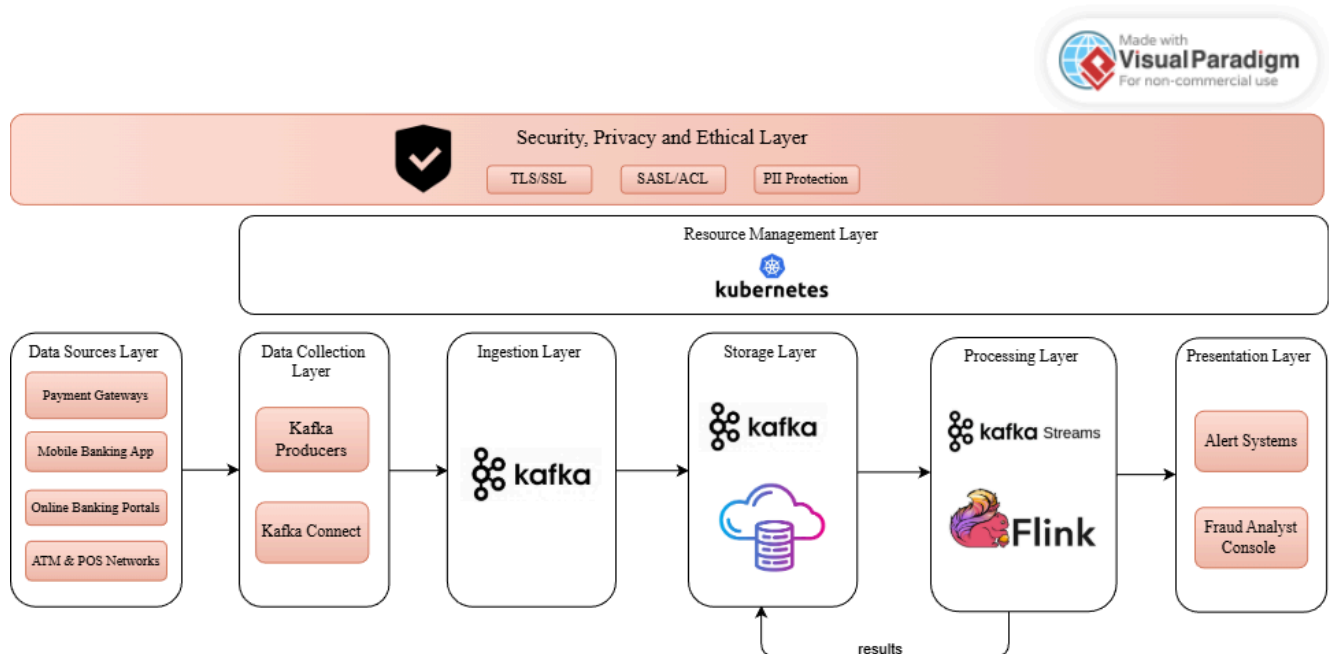


Figure 5 - Proposed Solution Architecture Diagram

3.1.1 Architecture Diagram Breakdown

The proposed architecture is an event-driven, hybrid system that is intended to satisfy the requirement of sub-500 ms latency to detect fraud in a high-volume financial environment.

- **Data Sources Layer**

The data layer includes the various sources of financial transactions including payment gateways, mobile banking applications, online banking portals and ATM networks. These sources provide a continuous, high-velocity stream of data holding structured information such as transaction ID, user ID, timestamp, and merchant details.

- **Data Collection Layer**

Apache Kafka is used to implement the data collection layer which is the main component for data ingestion. Through Kafka Producers, the primary agent that serializes and publishes data to Kafka topics, data is collected from different sources. Kafka is selected due to its distinct ability to handle high-velocity and large-volume data streams. Its durability is achieved by its

distributed, append-only commit log architecture, designed to store data on disk in a segmented, replicated format across a cluster of brokers. It is a file based log storage that has high throughput during sequential read and write, fundamentally different from random-access database, optimal for streaming data. Its scalability can be demonstrated in handling hundreds of thousands of writes per second, ideally suited for peak loads of a major international bank.

- **Ingestion Layer**

Apache Kafka is fulfilling the ingestion layer, serving as a message bus that connects all architecture components. It provides a durable, buffered storage for on-flight event streams, guaranteeing data is not lost during processing stages and enabling data replay in case of processing logic changes or failure recovery. Kafka Streams and the stateful processing engine (Apache Flink) interaction through Kafka topics reflect its role as the central integration backbone. This design ensures loose coupling where any component can be upgraded, scaled, or even fail without causing the failure of the entire system; which is essential in ensuring continuous operational integrity.

- **Storage Layer**

The architecture employs a two-strategy approach for storage, which is optimized to various stages of the data lifecycle. Apache Kafka is a temporary, high performance storage layer of short-term for real-time data in motion to facilitate stream processing and short-term replay. It stores data as a distributed commit log structure on the local disks of its brokers, which is different from traditional databases and file systems. For long-term historical storage, to store the processed results and raw transactions essential for model retraining, compliance, and batch analytics, data can be transferred to a data lake such as Cloud Storage. This separation is deliberate; as using a distributed file system in real-time processing would impose unacceptable latency for random I/O operations. The system uses Kafka for real-time capabilities and a cost-efficient cloud store for historical data, thus optimizing system performance and cost-efficiency.

- **Processing Layer**

The processing layer serves as the core analytical engine and is designed as a two-stage pipeline to compromise between latency and analytical depth, using two open-source tools. The initial phase of the processing path uses Kafka Streams to perform stateless transformations, including data validation, enrichment, and simple high priority rule execution.

It is well-suited due to its native integration with Kafka allowing low-latency processing with minimal overhead; eliminating the need for a separate processing cluster. Apache Flink is used for the processing intelligent path as it offers better stateful stream processing and complex event processing(CEP). In contrast to other stateless models, Flink is capable of maintaining and updating application state with high reliability even during failure events due to its distributed checking point mechanism. Its streaming model(one record at a time) guarantees a very low latency making it more suitable for this task compared to micro-batch-oriented frameworks such as Spark Streaming, where sub-second response is essential

- **Resource Management Layer**

Kubernetes automates the deployment, scaling and management of containerized applications. It would manage Kafka Streams, Flink job manager, task managers and microservices. This offers inherent scalability as Kubernetes will automatically scale processing pods up or down based on CPU load or Kafka topic lag. It improves fault tolerance by restarting failed containers automatically and rescheduling them to healthy nodes, providing high availability necessary for mission-critical financial systems.

- **Presentation Layer**

The presentation layer duty is to provide end-users and systems with insights and actions. This includes real-time dash boards representing system health, fraud trends, queues of medium-risk transactions for analyst reviews. Actions taken from processing layer outputs, such as alerting services for SMS/email notifications or transaction blocking in case of high risk transactions, and case management system for fraud analysts.

- **Security, Privacy and Ethical Layer**

A cross-cutting concern integrated within all layers. Security is enforced by TLS/SSL for encrypting data in transit between all components, authentication for access control to Kafka, Flink clusters, and Kubernetes by mTLS/SASL frameworks. Privacy is maintained by data tokenization of Personally Identifiable Information (PII) within stream processing.

3.1.2 Technologies Breakdown

Table 2 - Proposed Solution Technologies Breakdown

Aspect	Apache Kafka 	Kafka Streams 	Kubernetes 	Apache Flink 
Architecture	Distributed streaming platform, serving as a messaging system built around commit log. Consist of Brokers (servers) to manage Topics (messages categories). Topics are split into partitions for parallelism and scalability. Use ZooKeeper for cluster coordination and metadata managment .	Client library, provided by Apache Kafka, that integrates directly with the application. Each application is a standalone stream processor that communicates with Kafka clusters.	Container system with a client-server architecture consisting of a master node (handle API server, scheduler, control manager, storage) and working nodes (run application containers in Pods).	Stream processing framework. Distributed dataflow engine with JobManager (master node) that coordinated the distributed executions, and TaskManagers (slave nodes) that execute the dataflow tasks.
Programming Language	Java and Scala	Java	YAML/JSON Kubectl CLI	Java, Scala, Python
Query Language	API-driven	Domain Specific Language built in Java.	None	Apache Flink SQL for relational queries.

		Processor API		DataStreams and DataSet APIs for Java/Scala.
Data Storage / Processing Model	Log files on disk. Data stored as sequence of messages with offsets.	Processes data record by record basing through Kafka topics	Works as a resource manager scheduling and managing containerized applications.	Flink Table Store (dynamic tables for both streaming and batch processing)
	Acts as a real-time data bus. Producers publish messages to topics and consumers pull messages at their own pace.	Use embedded state stores (RockDB) for table operations.		True Streaming Processing one event at a time. Distributed, checkpointed state backend (e.g., RocksDB) to maintain large, durable state.
Main Features	High throughput. Fault tolerance by replication. Horizontal Scalability. Native Stream Processing with Kafka Streams.	Seamless integration with Apache Kafka. Fault tolerant local state storage. Exactly once processing semantics. Support event-time processing and windowing.	Automated deployment. Restart failed containers; self healing.	Low latency with high throughput. Complex windowing handling. Robust fault tolerance with distributed snapshots (checkpoints). Powerful state management; built-in support for Complex

				Event Processing (CEP) and ML.
Advantages	Extreme high performance and scalability. Durable storage prevents data loss.	Simple to deploy and operate. No external cluster management. Very low latency for stateless stateful operations.	High availability and resilience. Easy deployment. Portability across cloud and on-premises environments	Superior stateful processing capabilities. High performance for complex streaming workloads.
Limitations	Operational Complexity. Not designed for complex queries/transactional workloads.	Limited to the Kafka ecosystem. Java Centric. Not suitable for complex stateful operations. Scaling is limited to the number of Kafka topic partitions.	Complexity to manage and set up securely.	Requires separate clusters to manage. Operational overhead.
Source	https://www.geeksforgeeks.org/apache-kafka/apache-kafka/ .	https://www.geeksforgeeks.org/apache-kafka/what-is-apache-kafka-streams	https://www.geeksforgeeks.org/develops/kubernetes-architecture/	https://www.geeksforgeeks.org/cloud-computing/what-is-apache-flink/

3.2 Components Interaction

3.2.1 Workflow Diagram

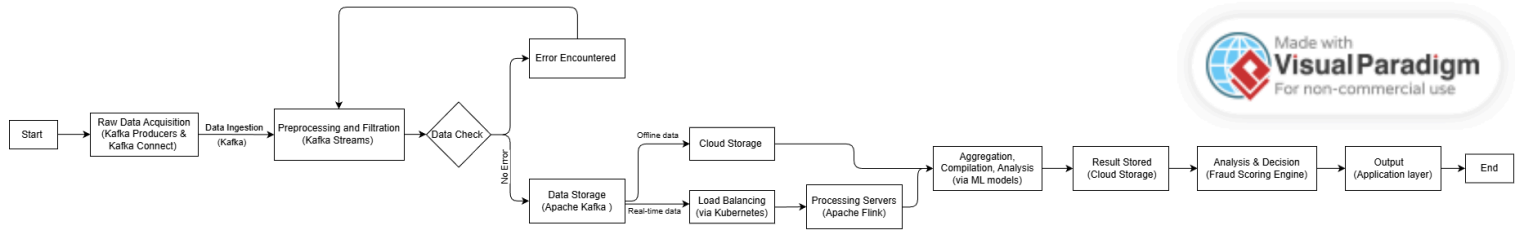


Figure 6 - Proposed Solution Workflow Diagram

The workflow starts with raw data acquisition from various financial sources including payment gateways, mobile banking applications, and ATM networks. These sources constantly produce transaction data with critical information like transaction amounts, user identifiers, timestamps, and geographic location.

Data collection layer utilizes Kafka Producers receiving events from custom applications (microservers, apps), and Kafka Connector receiving data from external systems (databases, legacy systems, cloud services). The system remains in a standby state during low activity periods, retaining connection pools and authentication sessions to guarantee instant responsiveness in case of transaction volume increase.

Upon ingestion, Kafka Streams applications begin instant preprocessing and filtration. It starts performing critical data cleaning functions as validation of data format, mandatory fields and transaction amounts against business rules. Kafka Streams is designed to work in a continuously running configuration, where each node has limited local state of windowed operations. In periods of low activity, applications are resource efficient by lowering checkpoint frequency without compromising the availability to instantly scale processing as transaction volume rises.

Transactions with no error moves to the data storage layer where Apache Kafka serves as a durable buffer and intelligent router. Kafka topic partitioned structure organizes data based on key attributes (user ID or transaction type), allowing parallel processing. The system automatically assigns data to real-time and offline streams based on predefined latency requirements. Kafka topics contain real-time data with short retention periods (1-7 days) optimized for low latency access, offline data is archived to long term cloud storage for historical analysis and model retraining. Kafka resumes replication and log compaction activities during idle periods to ensure data integrity and optimize storage usage.

The core analytic engine is at the processing layer driven by Apache Flink. Kubernetes operated load balancing by dynamically distributing Flink task managers across available compute resources, providing horizontal scaling in peak loads and consolidating in low usage periods. Flink performs complex stateful computations, behavioral baseline calculation, and apply ML models for fraud probability scoring. If not processing any active transactions, Flink store operator states using distributed checkpoints, maintain watermark advancement for temporal operations, and optimizes memory management for fast state recovery when receiving new data.

Processed results are stored in cloud storage; analysis and decision consumes it for final fraud classification. Business rules are applied to Flink generated fraud scoring, assigning risk levels (low, medium, high) on transactions to determine necessary actions. The selection is carried out by the application layer where high risk transactions are blocked by real-time alert systems, medium risk transactions are sent to fraud analyst consoles for manual reviews, low risk transactions are processed by banking APIs as normal. Components remain connected and update dashboard caches during system quiet periods to monitor operational metrics and performance.

The entire system operates under Kubernetes orchestration, constantly monitoring resource consumptions, autoscaling components based on Kafka topics log metrics; providing high availability by pod replication and failure recovery mechanism.

4. Challenges, findings and recommendations

Although adopting a real-time fraud detection system is technically feasible, there are several operational challenges that require practical solutions. A primary issue is that financial fraud is naturally dynamic; criminal strategies keep evolving causing model drift, ML models lose accuracy over time (Sunandoroy, 2025). This does not only require initial deployment, but ongoing maintenance to ensure continuous efficiency. System reliability is another critical challenge, as any component downtime might lead to undetected fraudulent transactions and direct financial losses. False positives require careful consideration, many false flags might frustrate legitimate customers and overwhelm fraud analysis teams with false alerts, eventually causing system distrust (Infosys Limited, 2025).

The proposed hybrid architecture, using Kafka Streams for rapid filtering and Apache Flink for complex analysis, balances speed and analytical depth successfully. The design enables efficient resource allocation and achieves sub-500ms latency requirement. Apache Kafka's main role as a durable message bus is essential for system resilience, guaranteeing persistence of data despite any component failure. Containerized approach with Kubernetes orchestration provides the scalability and high availability required, allowing the system to handle fluctuating transaction volumes effectively.

To address these challenges and enhance existing and proposed solutions, a regular weekly model retraining schedule should be implemented including the latest transaction data and newly validated fraud patterns to overcome model drifts. To mitigate false positives, establishing a straightforward feedback mechanism for fraud analysts to label false alarms while using the data to automatically tune model sensitivity thresholds. To guarantee business continuity, deployment of a fully replicated disaster recovery environment in another cloud data center is necessary to sustain operations during unexpected outages. A specific operation team is assigned to actively monitor the system will prevent minor problems turning into significant events. These measures will ensure system robustness, accuracy and reliability after initial implementation, providing long-term value in the battle against financial fraud.

References

- ACFE Press Release (2024) Acfe. Available at: <https://www.acfe.com/about-the-acfe/newsroom-for-media/press-releases/press-release-detail?s=2024-Report-to-the-Nations>
- ReenBit-admin (2025) Batch vs Stream Processing: Understanding the Trade-offs, Reenbit. Available at: <https://reenbit.com/batch-vs-stream-processing-understanding-the-trade-offs/>
- Moon, I.T. et al. (2022) “Towards the Advancement of Cashless Transaction: A Security Analysis of Electronic Payment Systems,” *Journal of Computer and Communications*, 10(7), pp. 103–129. Available at: <https://doi.org/10.4236/jcc.2022.107007>.
- Synthetic Identity Fraud (SIF) (2022) Hybrid and Cyber Risks. Available at: <https://www.synthetic-identity-fraud.com/>
- Mishra, S. (2023) ‘Scaling Rule Based Anomaly and Fraud Detection and Business Process Monitoring Through Apache Flink’, *International Journal of AI, BigData, Computational and Management Studies*, 4. Available at: <https://doi.org/10.63282/3050-9416.ijaidcms-v4i1p111>.
- Ververica (2024) What is Apache Flink? Complete Guide to Stream Processing, Ververica. Available at: <https://www.ververica.com/what-is-apache-flink>
- Bhargav Vadgama (2025) ‘Real-Time Fraud ML on Spark Structured Streaming: Micro-Batch vs. Continuous Processing’, *Utilitas Mathematica*, 122(2), pp. 856–879. Available at: <https://utilitasmathematica.com/index.php/Index/article/view/2786>
- Kishore, N. (2025) ‘AI-DRIVEN REAL-TIME FRAUD DETECTION USING KAFKA STREAMS IN FINTECH’, *International Journal of Applied Mathematics*, 38(6s), pp. 770–782. Available at: <https://doi.org/10.12732/ijam.v38i6s.433>.
- GeeksforGeeks (2017) *What is Apache Kafka?*, *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/apache-kafka/apache-kafka/>.
- GeeksforGeeks (2023) *What is Apache Kafka Streams?*, *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/apache-kafka/what-is-apache-kafka-streams>
- GeeksforGeeks (2022) *Kubernetes Architecture*, *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/devops/kubernetes-architecture/>
- GeeksforGeeks (2022b) *What is Apache Flink?*, *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/cloud-computing/what-is-apache-flink/>
- Infosys Limited (2025) *Fraud Detection Techniques and Challenges | Infosys BPM*, *Infosysbpm.com*. Available at: <https://www.infosysbpm.com/glossary/fraud-detection.html>.

- Sunandoroy (2025) *Fraud Detection in Financial Transactions : Challenges and Innovations - SUNANDO ROY - On Banking, Finance and Society, SUNANDO ROY - On Banking, Finance and Society - My Blog*. Available at:
<https://sunandoroy.org/2025/01/29/fraud-detection-in-financial-transactions-challenges-and-innovations/>

Figures

1. BANKiQ (2024) BANKiQ. Available at:
<https://bankiq.co/understanding-real-time-payments-fraud-detection-everything-you-needed-for-fortifying-payment-security/>.
2. From Aligned to Unaligned Checkpoints - Part 1: Checkpoints, Alignment, and Backpressure (2020) Apache.org. Available at:
<https://flink.apache.org/2020/10/15/from-aligned-to-unaligned-checkpoints-part-1-checkpoints-alignment-and-backpressure/>
3. Sarda, M. (2022) Batch Processing vs Stream Processing in Microsoft Azure. Available at:
<https://k21academy.com/microsoft-azure/data-engineer/batch-processing-vs-stream-processing/>.
4. An overview of Apache Kafka use cases | Redpanda (2025) Redpanda.com. Available at: <https://www.redpanda.com/guides/kafka-use-cases>.
5. Own Work. Figure created using Visual
<https://online.visual-paradigm.com/share.jsp?id=333136353236342d3130>
6. Own Work. Figure created using Visual Paradigm
<https://online.visual-paradigm.com/share.jsp?id=333136353236342d3131>

Marking Rubrics

BIG DATA TECHNOLOGIES ITS66904 Assignment Marking Rubrics (SEPTEMBER 2025)				
Criteria	Score			
	Total Marks: 100 (will be capped at 30%)			
	Excellent	Good	Average	Poor
	$\geq 80\%$	$< 80\%$ to $\geq 65\%$	$< 65\%$ to $\geq 50\%$	$< 50\%$
Task 1: Research and Analysis (50 Marks)				
Q1a. Literature Review of Existing Solutions	Comprehensive review; Equal or more than 3 scholarly, reliable sources are reviewed and discussed; deep insights into approaches & technologies.	Adequate review; Equal or more than 2 scholarly, reliable sources are reviewed and discussed; good but less detailed coverage.	Limited review; minimal sources; superficial discussion.	Missing/irrelevant; no scholarly basis.
Q1b. Critical Analysis of Strengths & Limitations	Insightful, balanced critique; strong evidence; clear link to velocity & volume.	Solid critique; mostly relevant; some evidence.	Basic mention of strengths/limits; little depth or evidence.	Missing/weak analysis; not linked to case.
Q2a. Proposed Architecture & Tools	Suitable architecture + 2 tools; strong justification with references.	Appropriate choice; justification mostly sound.	Basic description; weak or generic justification.	Missing/irrelevant proposal.
Q2b. Explanation of system functions in practice:	Clear, accurate, and logical explanation of functions and interactions.	Reasonable and logical explanation of functions and interactions with a few minor mistakes.	Limited explanation of functions and interactions with minor mistakes.	Weak/absent explanation and/or major mistakes.